Fixed-Point vs. Floating-Point in IIR Filtering Does the Format Choice Actually Matter?

1st Sepehr Farrokhi
Hanze University of Applied Sciences
s.farrokhi.koochesfahani@st.hanze.nl

2nd Amin Javan
Hanze University of Applied Sciences
ma.javan@st.hanze.nl

3rd Armin Siavashi
Hanze University of Applied Sciences
a.siavashi@st.hanze.nl

***Abstract*—When designing a DSP system, the choice between fixed-point and floating-point arithmetic directly affects accuracy, memory consumption, and hardware cost. This assignment evaluates both dimensions using a 4th-order Butterworth IIR low-pass filter as a testbed. Two fixed-point formats (Q3.13 and Q8.8) and two floating-point formats (`float16` and `float32`) are simulated and compared against a `float64` high-precision reference. not a true ground truth, but a sufficiently accurate baseline for quantifying representational error. Accuracy is measured via MSE and SNR; efficiency is assessed through memory footprint and a qualitative hardware cost analysis. The most striking finding is that format selection within fixed-point matters more than the fixed vs. floating-point distinction itself: Q3.13 achieves 44.87 dB SNR while Q8.8 reaches only 13.8 dB, despite identical 16-bit word length and 2000 bytes of memory. Fixed-point offers lower hardware cost and power consumption on embedded targets; floating-point trades those efficiencies for significantly higher precision and simpler programming.**

***Index Terms*—fixed-point, floating-point, IIR filter, quantisation error, Q-format, IEEE 754, DSP**

I. Introduction

Every DSP system encodes numbers in finite binary words. Whether those words use fixed-point or floating-point arithmetic determines accuracy, memory consumption, and hardware cost. three things that pull in different directions depending on the application. This decision is made early and is difficult to reverse: a processor either has a floating-point unit or it does not.

Neither paradigm is universally better. Fixed-point dominates in hearing aids, sensor nodes, and motor controllers where power and cost are the binding constraints [2]. Floating-point is preferred wherever precision matters and hardware cost is secondary [3]. What is less obvious is that the choice of format *within* fixed-point. how many bits go to the integer part versus the fractional part. can matter more than the fixed vs. floating-point distinction itself. Texas Instruments note that Q-format selection is a critical design decision that directly impacts filter accuracy [3]. This assignment tests that claim directly.

We evaluate four representations applied to a 4th-order Butterworth IIR low-pass filter: Q3.13 (13 fractional bits), Q8.8 (8 fractional bits), `float16` (10 mantissa bits), and `float32` (23 mantissa bits). An IIR topology was chosen because its recursive feedback re-introduces rounding errors on every output cycle, making it a more sensitive testbed than a non-recursive FIR design [1]. For each format we measure MSE, SNR, and memory footprint against a `float64` high-precision baseline.

We address two research questions: (1) What are the main characteristics, advantages, and disadvantages of fixed-point and floating-point representations for IIR filter processing, in terms of accuracy and efficiency? (2) What quantisation errors are observed for Q3.13, Q8.8, `float16`, and `float32`?

II. Background

A. Fixed-Point Representation

A Q$m.n$ number stores m integer bits and n fractional bits in a signed $(m+n)$-bit word [1]. Resolution is 2^{-n} uniformly across the range $[-2^{m-1},\ 2^{m-1}-2^{-n}]$. Because the binary point never moves, the programmer must choose a format whose range covers all signal values before filtering begins.

Q3.13 uses 13 fractional bits in a 16-bit word: range $[-4, 4)$, resolution $\approx 1.2 \times 10^{-4}$. It is the minimum safe format for this filter, whose coefficients span $[-1.32, 2.0]$; Q1.15 was initially considered but rejected because its range of $[-1, 1)$ cannot represent these coefficients, causing integer overflow and complete filter corruption. **Q8.8** uses 8 integer and 8 fractional bits: range $[-128, 128)$, resolution $\approx 3.9 \times 10^{-3}$. 32 times coarser than Q3.13. Both use identical 16-bit words and identical memory; only the binary point position differs.

On hardware, integer multiply-accumulate units are smaller and more power-efficient than floating-point units, making fixed-point the standard choice for microcontrollers without an FPU [2]. The ARM Cortex-M0 is a widely used example of this class of device. The precision cost is that coefficient quantisation in IIR filters shifts pole locations, which alters the frequency response. Poles close to the unit circle. typical in selective low-pass designs. are especially sensitive to this shift [1].

B. Floating-Point Representation

IEEE 754 floating-point encodes a value as $(-1)^s \times 1.M \times 2^{E-\text{bias}}$, where s is the sign, M the mantissa, and E the biased exponent [4]. The exponent adapts to the signal magnitude automatically, giving roughly constant *relative* precision across a wide dynamic range. no manual scaling required.

`float16` provides 10 mantissa bits (≈3 decimal digits, max 65504). `float32` provides 23 mantissa bits (≈7 decimal digits, max $\approx 3.4 \times 10^{38}$). The efficiency cost is hardware area: a 32-bit FPU occupies roughly 10–40× more silicon than an

equivalent integer unit and draws more power [2]. Floating-point precision comes at a real hardware price.

III. Simulation Setup

This assignment requires observing quantisation errors across 2+2 representations. An IIR filter was chosen as the application because its recursive structure re-introduces rounding errors on every output sample, making quantisation effects more pronounced and easier to observe than in a non-recursive FIR design [1]. A 4th-order Butterworth low-pass filter was selected for its maximally flat passband, ensuring that any deviation from the reference is caused by quantisation rather than pre-existing ripple. SOS format was used instead of direct-form coefficients because it reduces numerical sensitivity in higher-order designs [1].

The filter was designed with cut-off at 100 Hz using `scipy.signal.butter` at $f_s = 1000$ Hz. The input is 1000 samples of a 50 Hz sine wave with added out-of-band components, placing the in-band signal clearly in the passband and providing a visible stopband for error analysis. The four formats were chosen to give a symmetric comparison: Q3.13 and Q8.8 represent fine and coarse fixed-point using identical 16-bit words; `float16` and `float32` represent low and high precision floating-point.

Fixed-point filtering used the `spfpm` library [5], which provides behavioural simulation of Q-format arithmetic including rounding and overflow handling. Floating-point filtering used `scipy.signal.sosfilt` [6] with `NumPy` dtype casting. All outputs are compared against a `float64` baseline. not a mathematically exact reference, but with 52 mantissa bits its rounding error is negligible relative to the formats under test [4]. Error metrics are:

$$\mathrm{MSE} = \frac{1}{N}\sum_{n=0}^{N-1}(y_{\mathrm{ref}}[n]-\hat{y}[n])^2, \quad \mathrm{SNR} = 10\log_{10}\left(\frac{\sigma^2_{y_{\mathrm{ref}}}}{\mathrm{MSE}}\right)\mathrm{dB} \tag{1}$$

Memory is reported as the byte size of the signal array. Execution time is not compared: `spfpm` runs in pure Python and reflects simulation overhead, not hardware performance. On a general-purpose x86 CPU, floating-point executes faster due to dedicated SIMD units; on an embedded target without an FPU, fixed-point integer operations would be faster, consistent with Analog Devices [2].

IV. Results

TABLE I
Quantisation error metrics for the IIR filter (1000 samples, $f_c = 100$ Hz).

Format	MSE	SNR (dB)	Memory
Q3.13 (fixed)	4.04×10^{-6}	44.87	2000 B
Q8.8 (fixed)	5.12×10^{-3}	13.84	2000 B
`float16` (float)	3.03×10^{-9}	76.12	2000 B
`float32` (float)	3.69×10^{-15}	135.27	4000 B

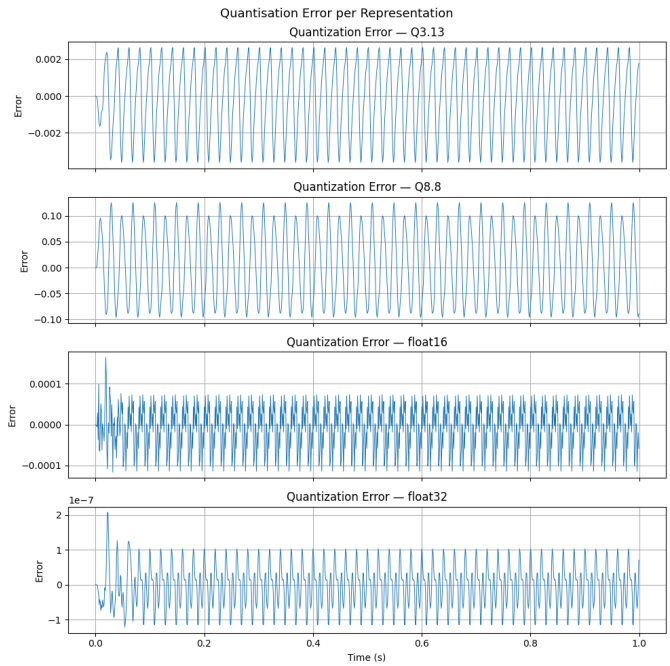

Fig. 1. Per-sample quantisation error $e[n] = y_{\mathrm{ref}}[n] - \hat{y}[n]$ for all four formats. Note the different y-axis scales: Q8.8 error is ± 0.10; float32 error is at the 10^{-7} level.

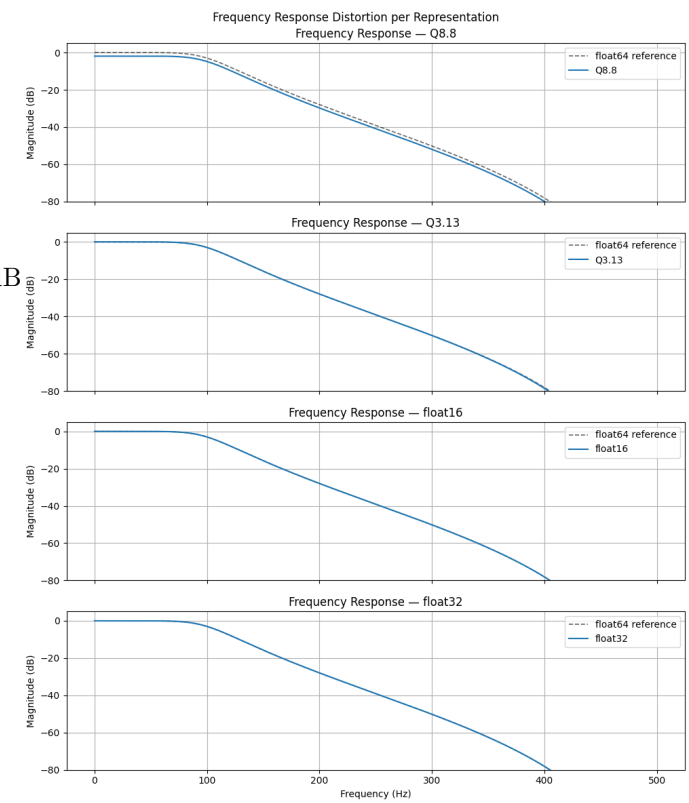

Fig. 2. Frequency response magnitude for each quantised SOS filter vs. the float64 reference (black dashed). Q8.8 shows visible deviation in both passband and stopband.

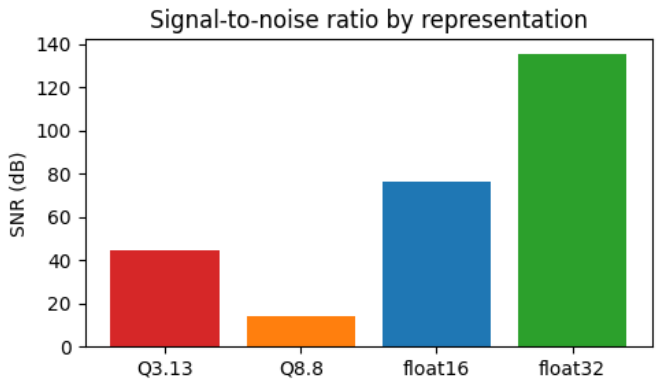

Fig. 3. SNR (dB) across all four representations. Higher is better.

V. Discussion

A. RQ1: Characteristics, Advantages, and Disadvantages

Fixed-point and floating-point differ along two axes: efficiency and accuracy. On efficiency, fixed-point wins for embedded targets. A 16-bit integer multiplier is smaller and more power-efficient than a 32-bit FPU; Analog Devices estimate fixed-point DSP cores cost roughly half as much and draw significantly less power [2]. For battery-powered devices that is often the deciding factor. IIR filters are more sensitive to fixed-point quantisation than FIR filters because recursive feedback accumulates rounding error on every sample, whereas FIR errors are bounded by the filter length [1].

On accuracy, floating-point's adaptive exponent means precision scales with signal magnitude automatically. no manual scaling, no hard overflow for moderate signal levels. Fixed-point requires bit allocation to be decided upfront. In IIR filter design this matters because the same coefficients that define the filter's poles go through quantisation. A coarsely quantised coefficient shifts a pole's location, which changes the frequency response, as Q8.8 demonstrates in Fig. 2 [1].

The within-fixed-point comparison makes the allocation cost concrete. Q3.13 and Q8.8 use identical memory (2000 bytes) but differ by a factor of 1,270 in MSE. Q8.8 allocates 8 bits to an integer range ($[-128, 128)$) that a normalised 100 Hz sine wave never uses, leaving only 8 fractional bits for a signal that needs fine resolution. Q3.13 allocates 13 bits to fractional precision. the minimum safe match for this filter's coefficient range [3]. `float16` avoids this problem entirely: at the same 2000 bytes it achieves 76 dB SNR versus Q3.13's 44.87 dB, because its exponent adapts precision sample-by-sample [4].

B. RQ2: Observed Quantisation Errors

The accuracy ranking is `float32` $\gg$ `float16` $>$ Q3.13 $\gg$ Q8.8 (Table I). Three observations are worth noting.

First, the *pattern* of error differs structurally between fixed and floating-point (Fig. 1). Fixed-point error is periodic and uniform in amplitude. each quantisation step is the same size, so the error waveform repeats with the signal period. Floating-point error is irregular and signal-dependent, because the step size scales with magnitude [1]. Both are forms of quantisation noise, but they have different spectral properties that matter for audio applications.

Second, the frequency response (Fig. 2) shows that Q8.8 coefficient quantisation shifts the filter's poles enough to produce visible deviation from the reference in both passband and stopband. Q3.13, `float16`, and `float32` remain close to the reference. This confirms that IIR coefficient wordlength directly determines filter shape accuracy [1].

Third, `float32` at 135 dB SNR is effectively indistinguishable from the `float64` baseline for this filter. It uses twice the memory of the 16-bit formats (4000 vs. 2000 bytes), but on any system where an FPU is present, it is the appropriate default.

VI. Conclusion

The fixed vs. floating-point decision is a genuine efficiency-accuracy tradeoff. Fixed-point is the right choice for embedded targets without an FPU: Q3.13 achieves 44.87 dB SNR at 2000 bytes with no floating-point hardware required. `float32` is the right choice when an FPU is present, delivering 135 dB SNR. effectively indistinguishable from the double-precision baseline. at twice the memory.

The more important finding is within fixed-point. Q8.8 and Q3.13 cost the same in memory and hardware word length, yet Q8.8 is unsuitable for this IIR filter application: its 31 dB SNR deficit relative to Q3.13 stems entirely from a mismatched integer range. Format selection within fixed-point is not a secondary concern. it is the primary one. Future work should validate the qualitative efficiency claims made here by executing these formats directly on embedded hardware, for example comparing runtime on an ARM Cortex-M0 without an FPU versus a Cortex-M4 with an FPU. Additionally, extending the simulation to higher-order or Chebyshev filters would further expose the vulnerability of fixed-point poles to coefficient quantisation [1].

References

[1] S. W. Smith, *The Scientist and Engineer's Guide to Digital Signal Processing*, ch. 28: "Fixed versus Floating Point," California Technical Publishing, 1997. [Online]. Available: http://www.dspguide.com/ch28/4.htm

[2] Analog Devices, "Fixed-Point vs. Floating-Point Digital Signal Processing," *Analog Devices Technical Article*, 2007. [Online]. Available: https://www.analog.com/en/resources/technical-articles/fixedpoint-vs-floatingpoint-dsp.html

[3] G. Frantz and R. Simar, "Fixed-Point vs. Floating-Point Digital Signal Processing," Texas Instruments, Application Report SPRY061, 2007. [Online]. Available: https://www.ti.com/lit/wp/spry061/spry061.pdf

[4] Wikipedia, "IEEE 754," *Wikimedia Foundation*. [Online]. Available: https://en.wikipedia.org/wiki/IEEE_754

[5] R. W. Penney, "SPFPM: Simple Python Fixed-Point Module," *GitHub repository*, 2024. [Online]. Available: https://github.com/rwpenney/spfpm

[6] P. Virtanen et al., "SciPy 1.0: Fundamental Algorithms for Scientific Computing in Python," *Nature Methods*, vol. 17, pp. 261–272, 2020. [Online]. Available: https://rdcu.be/b08Wh